Maathrucare: An AI-Based Pregnancy Support System for Sri Lankan Mothers

INDIVIDUAL THESIS

Thirimanne D.P.B.

(IT22272386)

Individual component: AI-Assisted CTG Interpretation Module: Image-Based Fetal Heart Rate and Uterine Contraction Reconstruction with Fetal Well-Being Classification

Group ID: R26-IT-146

Report submitted in partial fulfilment of the requirements for the

Bachelor of Science (Honours) in Information Technology

Specialising in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: Thirimanne D.P.B.

Student ID: IT22272386

Signature: __Bimsara_____ Date: _____

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Supervisor: Dr. Kapila Dissanayake		
Co-supervisor: Mr. Gayan		

ABSTRACT

Cardiotocography (CTG) is the standard method for monitoring fetal heart rate (FHR) and uterine contractions (UC), yet its interpretation remains manual, hospital-bound and subject to substantial inter-observer variation. Pregnant women usually leave a clinic holding only a printed CTG strip, and existing automated interpretation systems require a digital signal taken directly from the monitor. This individual component of the Pregnancy Support System (group R26-IT-146) investigates whether a smartphone photograph of a CTG printout can be converted into clinically meaningful features and a fetal well-being category. A Design Science Research approach was used to build and evaluate a modular Python service that pre-processes the photograph, localises the graph region, reconstructs FHR and UC time series by trace segmentation, derives guideline-inspired features (baseline, short- and long-term variability, accelerations, decelerations and contraction statistics) and classifies the record as Normal, Suspect or Pathological with an XGBoost model. Paired image–signal data rendered from the open CTU-UHB intrapartum database were used to evaluate reconstruction, and the UCI Cardiotocography dataset (2,126 records) was used to train and compare four classifiers under stratified five-fold cross-validation. XGBoost achieved the best macro-F1 of 0.917 on tabular features. For image-based reconstruction, connected-component trace segmentation produced a lower mean feature-bridge error than a purpose-trained U-Net (6.56 versus 9.18) and was therefore selected for production even though the U-Net achieved better raw signal agreement. A synthetic round-trip test suite further revealed a failure mode in which full-width ruling lines can be mistaken for the trace. The work contributes a three-level evaluation framework (signal, feature and classification) that keeps tabular accuracy separate from end-to-end image performance, and a deployable FastAPI service with explicit signal-quality gating. End-to-end clinical accuracy could not be claimed because window-level clinical labels were unavailable, and this is identified as the principal direction for future work.

Keywords: cardiotocography, fetal heart rate, image-based signal reconstruction, XGBoost, mobile health

ACKNOWLEDGEMENTS

I am sincerely grateful to my supervisor, Dr. Kapila Dissanayake, for his continuous guidance, constructive feedback and encouragement throughout this research, and to my co-supervisor, Mr. Gayan, for valuable advice on the direction of the work.

I thank my teammates in group R26-IT-146 for their collaboration in designing and integrating the Pregnancy Support System, and the Department of Information Technology of the Sri Lanka Institute of Information Technology for providing the academic environment and facilities required for this project. I also acknowledge the medical professionals who shared practical knowledge about CTG usage and maternal-care workflows, and the creators of the CTU-UHB Intrapartum Cardiotocography Database and the UCI Cardiotocography dataset for making their data openly available.

Finally, I thank my family for their patience and support throughout my studies.

TABLE OF CONTENTS

1 INTRODUCTION 1
1.1 Overall project context 1
1.2 Individual component context 2
1.3 Problem statement 2
1.4 Research gap 3
1.5 Aim and objectives 4
1.6 Research questions or hypotheses 4
1.7 Scope limitations and assumptions 5
1.8 Individual contribution 5
1.9 Thesis structure 6
2 LITERATURE REVIEW AND RELATED WORK 7
2.1 Review method and scope 7
2.2 Domain and theoretical foundations 7
2.2.1 CTG interpretation criteria 7
2.2.2 Clinical value and limitations of CTG 8
2.2.3 Technical foundations 8
2.3 Related research 8
2.4 Existing systems products and techniques 10
2.5 Gap synthesis and conceptual framework 10
3 RESEARCH METHODOLOGY 11
3.1 Research design 11
3.2 Data participants or study materials 11
3.3 Experimental or study procedure 12
3.3.1 Classifier comparison 12

3.3.2 Graph detection 13
3.3.3 Signal reconstruction and U-Net training 13
3.3.4 Feature bridge and end-to-end runs 13
3.4 Measures and evaluation metrics 14
3.5 Data analysis 15
3.6 Ethics privacy security and responsible AI 15
3.7 Reproducibility and research artefacts 15
4 PRODUCT AND SOLUTION DESIGN 17
4.1 Requirements 17
4.2 Component architecture within the complete solution 18
4.3 Design alternatives and decisions 19
4.4 Detailed design 20
4.4.1 Pre-processing and graph detection 20
4.4.2 Trace reconstruction 20
4.4.3 TraceUNet 21
4.4.4 Feature extraction 21
4.4.5 Classification and response schema 22
4.5 Standards and professional practice 22
5 IMPLEMENTATION INTEGRATION AND TESTING 23
5.1 Development environment and technology stack 23
5.2 Component implementation 23
5.2.1 Configuration-driven analyser 23
5.2.2 Quality gating 24
5.2.3 Classifier training and persistence 24
5.2.4 Service interface 24
5.3 Integration with the other components 24

5.4 Verification and software testing 25
5.5 Deployment and operation 27
6 RESULTS AND VALIDATION 28
6.1 Research results 28
6.1.1 Classifier comparison (RQ1) 28
6.1.2 Signal reconstruction and the feature bridge (RQ2, RQ3) 29
6.1.3 Failure conditions (RQ4) 30
6.2 Product and technical results 30
6.3 User or stakeholder validation 31
6.4 Integration results 31
6.5 Summary of findings 32
7 DISCUSSION 33
7.1 Interpretation of findings 33
7.2 Comparison with literature baselines and existing solutions 33
7.3 Research contribution 34
7.4 Product contribution 34
7.5 Limitations and threats to validity 34
7.6 Implications 35
8 COMMERCIALISATION AND SUSTAINABILITY 36
8.1 Value proposition and target users 36
8.2 Market and competitive position 36
8.3 Business or sustainability model 36
8.4 Scalability intellectual property and compliance 37
8.5 Contribution to the Sustainable Development Goals 37
9 CONCLUSION AND FUTURE DIRECTIONS 39
9.1 Conclusion 39

9.2 Achievement of objectives 39
9.3 Individual research and product contributions 39
9.4 Future directions 40
REFERENCES 41
GLOSSARY 44
APPENDICES 45
Appendix A Extended results and technical specifications 45
Appendix B Academic integrity and AI use disclosure 46

LIST OF FIGURES

Figure 1.1: Overall architecture of the Pregnancy Support System and the position of the individual component 2

Figure 3.1: Three-level evaluation framework used in this study 14

Figure 4.1: Internal processing pipeline of the CTG interpretation module 18

Figure 4.2: Architecture of the TraceUNet FHR segmentation network 21

Figure 5.1: Synthetic round-trip test: (a) rendered strip; (b) correct reconstruction; (c) failure in which a border line is selected as the trace 27

Figure 6.1: Gain-based feature importance of the production XGBoost classifier (top 12 of 21 features) 29

Figure 6.2: Mean absolute feature error for the two leading FHR extractors 30

LIST OF TABLES

Table 2.1: Comparison of representative related work with this study 9

Table 3.1: Datasets used in this study 11

Table 3.2: Evaluation metrics 14

Table 3.3: Research artefacts 16

Table 4.1: Functional requirements of the CTG interpretation module 17

Table 4.2: Non-functional requirements 17

Table 4.3: Design alternatives and decisions 19

Table 5.1: Technology stack 23

Table 5.2: Service endpoints 24

Table 5.3: Unit and round-trip test results 25

Table 6.1: Five-fold cross-validated classifier performance on UCI tabular features (n = 2,126) 28

Table 6.2: Comparison of FHR extractors on rendered CTU-UHB samples 29

Table 6.3: Summary of findings by research question 32

Table 8.1: Estimated development cost for the component (LKR) 36

Table 9.1: Achievement of objectives 39

Table A.1: Gain-based importance of all 21 input features of the production XGBoost model 45

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AC	Accelerations (UCI feature)
AI	Artificial Intelligence
ALTV	Percentage of time with abnormal long-term variability
API	Application Programming Interface
ASTV	Percentage of time with abnormal short-term variability
BCE	Binary Cross-Entropy
bpm	beats per minute
CLAHE	Contrast-Limited Adaptive Histogram Equalisation
CNN	Convolutional Neural Network
CTG	Cardiotocography
CTU-UHB	Czech Technical University – University Hospital Brno (database)
CV	Cross-Validation
DSR	Design Science Research
FHR	Fetal Heart Rate
FIGO	International Federation of Gynecology and Obstetrics
IoU	Intersection over Union
JSON	JavaScript Object Notation
LTV	Long-Term Variability
MAE	Mean Absolute Error
ML	Machine Learning
NICE	National Institute for Health and Care Excellence
NSP	Normal, Suspect, Pathological (class label)
REST	Representational State Transfer
RMSE	Root Mean Square Error

Abbreviation	Meaning
SDG	Sustainable Development Goal
STV	Short-Term Variability
SVM	Support Vector Machine
UC	Uterine Contraction
UCI	University of California, Irvine (Machine Learning Repository)
XGBoost	Extreme Gradient Boosting
YOLO	You Only Look Once (object detector)

1 INTRODUCTION

1.1 Overall project context

Antenatal care is intended to raise the chances of identifying pregnancy complications early enough for intervention, and WHO recommends a minimum schedule of antenatal contacts for all pregnancies [1]. Electronic fetal monitoring with cardiotocography (CTG) is one of the most popular methods in that regard: a CTG machine is equipped with a Doppler transducer that captures the fetal heart rate (FHR) and a tocodynamometer that measures and records the pressure of the uterine contractions (UC) on a paper strip [2]. The clinician reads the paper strips for the baseline HR, variations, accelerations, decelerations, and the phase relationship between the decelerations and contractions to assess fetal oxygenation.

The Pregnancy Support System (group R26-IT-146) is a mobile-first platform that supports mothers and their health practitioners in monitoring pregnancy outside of hospital. As described in the group proposal, the full solution consists of: an app that mothers use to input their health data and upload documents; an AI-based engine that processes the CTG trace and scores it; a doctor-facing interface for review and input; and, an AWS cloud backend for storing records and enabling longitudinal trend analysis. See Figure 1.1 for how these parts connect. The end-users – mothers and doctors - simply use the app and the review interface, respectively. The AI services are available via a network API.

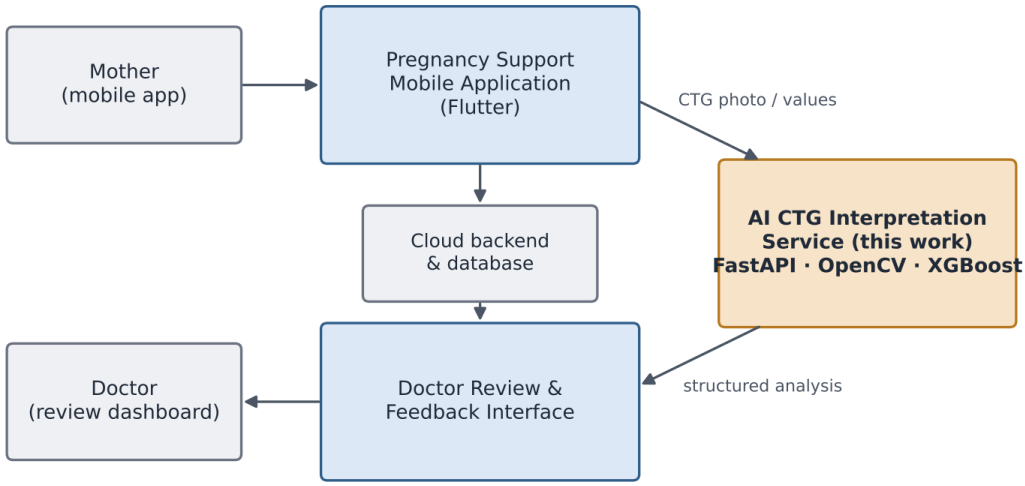


Figure 1.1: Overall architecture of the Pregnancy Support System and the position of the individual component

1.2 Individual component context

As you might expect, the core component of the system will be the AI-assisted CTG Interpretation Module. The module takes an image of a CTG printout captured on a phone camera (or CTG values entered manually) and, after analysis, outputs an object that contains: the identified region of the CTG graph; the interpreted FHR and UC signals; a set of interpretable CTG features; a verdict on the signal quality and a class-based interpretation of fetal well-being, along with class probabilities. This object is stored by the backend, displayed to the doctor for review and correction, and then transmitted in plain language back to the mother. This component was also implemented as a separate Python service (ctg-ai-service) with a FastAPI interface so as to enable development, evaluation and deployment independently of the mobile and web clients, while exposing a stable interface to them.

1.3 Problem statement

CTG interpretation are: 1) its subjectivity, CTG readers only fairly good at saying the same thing about the same trace, leading to the development of computerised interpretation programs, like Sis Porto [3] and consensus guidelines [2], [4]; 2) it can only be performed in hospital, the trace is read in a clinic and the mother goes home with little more than a sheet of paper that cannot be read; 3) computerised

interpretation programs only work on the monitor's digital signal and cannot be used on the monitor's digital signal and cannot be used on a paper trace.. For mothers who live at a distance from the nearest specialist, who are transferred from one facility to another, or those who wish to send an up-to-date CTG to her own obstetrician, the content of the printout is thus locked. The challenge this component is attempting to solve: how can a simple snapshot of a CTG printout, photographed using a regular smartphone in a less than ideal environment, be decoded robustly into quantitative CTG features and a category of fetal wellbeing that complements (not replaces) the review of a trained clinician?

1.4 Research gap

There has been an array of published work on machine-learning classification of CTG, but with nearly all starting from clean digital features or signals. The benchmark that has seen the most use is the UCI Cardiotocography dataset, which consists of 21 features extracted by the Sis Porto program [5], so published results do not inform us about the ability to recover those features from an image. Deep-learning work on the CTU-UHB database [6] also operated directly on the recorded FHR signal [7], [8]. Four specific gaps were identified from the literature reviewed in Chapter 2 that this component addresses.

- Image-to-signal reconstruction - a limited number of studies address the fidelity of FHR and UC signals that can be reconstructed from images of paper strips, and no paper-based image-to-signal reconstruction studies in the paper workflow are within the scope of the papers reviewed.

- Feature-level evaluation - the most common metric used to evaluate reconstruction techniques is pixel or signal error, which fails to reveal whether important features such as accelerations and decelerations persist after reconstruction.

- Honest reporting of end-to-end performance - tabular classification accuracy often masquerades as the accuracy of a complete image-based system, thus over exaggerating what such a system can achieve.

1.5 Aim and objectives

Aim. To design, implement and evaluate an AI-assisted CTG interpretation module that converts a smartphone photograph of a CTG printout into reconstructed FHR and UC signals, interpretable CTG features and a fetal well-being classification suitable for clinician review within the Pregnancy Support System.

Objectives.

- **O1** – Pre-process uploaded CTG photographs and automatically localise the graph region.
- **O2** – Reconstruct FHR and UC time series from the graph region and compare alternative extraction methods, including classical image processing and a U-Net segmentation network.
- **O3** – Derive guideline-inspired CTG features (baseline, variability, accelerations, decelerations and contraction statistics) from the reconstructed signals.
- **O4** – Train and compare machine-learning classifiers that assign a Normal, Suspect or Pathological category with class probabilities.
- **O5** – Evaluate the pipeline at signal, feature and classification level without conflating tabular and image-based results.
- **O6** – Expose the module as a service with signal-quality gating that the mobile application and doctor interface can consume.

1.6 Research questions or hypotheses

The objectives were operationalised as the following research questions:

- **RQ1** – Which supervised classifier gives the best macro-averaged F1-score for three-class CTG classification on established tabular CTG features?
- **RQ2** – How accurately can FHR and UC signals be reconstructed from CTG images, and how do classical trace segmentation and a learned U-Net compare?
- **RQ3** – Does better signal-level agreement translate into better recovery of clinically relevant CTG features?
- **RQ4** – Under what image conditions does the reconstruction pipeline fail, and can such failures be detected so that unreliable classifications are withheld?

The working hypothesis for RQ3 was that signal-level error alone is an insufficient selection criterion, and that the method with the lowest feature-level error should be preferred for clinical use even if it is not the best at signal level.

1.7 Scope limitations and assumptions

The component covers image pre-processing, graph localisation, FHR and UC reconstruction, feature extraction, classification and the service interface. The mobile user interface, authentication, data storage and the doctor dashboard belong to other group members and are referred to only where they interact with this component. The following limitations and assumptions apply:

- The output is **decision support** for a qualified clinician; it is not a diagnostic device and every result is reviewed by a doctor before it reaches the mother.
- Each analysis assumes a strip of approximately five minutes (300 s, resampled to 4 Hz) with the FHR panel occupying the upper 72% of the graph and the UC panel the lower 28%, and FHR and UC axes of 30–240 bpm and 0–100 units respectively. Printouts with other layouts or scales require configuration changes.
- Paired image–signal data were produced by rendering CTU-UHB recordings as images with augmentations; real photographs of hospital printouts with ground-truth signals were not available in sufficient numbers for training.
- The classifier is trained on the UCI dataset, whose labels describe whole records assessed by experts. Window-level clinical labels for the rendered CTU-UHB images were not available, so end-to-end image-to-class accuracy is not claimed.
- Manually entered values are accepted through the same feature schema but the main research effort concerns the image path.

1.8 Individual contribution

All work described in this thesis was carried out by the author as the individual component of the group project. The author's specific contributions are:

- the modular *ctg-ai-service* code base, including interchangeable graph detectors (OpenCV contour detector and an optional YOLOv8 detector), three classical

FHR extractors, a U-Net FHR extractor, three UC extractors and the end-to-end analyser;

- a lightweight TraceUNet segmentation network (472,257 parameters) trained with masks generated from ground-truth signals rather than from image thresholding;
- the feature-extraction module and the mapping of reconstructed features to the classifier's input schema;
- the training and comparison of logistic regression, random forest, support vector machine and XGBoost classifiers, and the persisted production model;
- a three-level evaluation framework (signal, feature bridge, classification), evaluation scripts, a ground-truth label audit and a unit and synthetic round-trip test suite;
- the FastAPI service with health, model-information and analysis endpoints and explicit signal-quality gating.

1.9 Thesis structure

Chapter 2 reviews CTG interpretation, computerised and machine-learning approaches and image digitisation, and synthesises the research gap. Chapter 3 describes the research design, data, procedure and evaluation metrics. Chapter 4 presents the requirements and the design of the component within the complete solution. Chapter 5 describes implementation, integration and testing. Chapter 6 reports the results and Chapter 7 discusses them against the literature, including limitations. Chapter 8 considers commercialisation and sustainability, and Chapter 9 concludes and outlines future work.

2 LITERATURE REVIEW AND RELATED WORK

1.2 Review method and scope

The review was carried out in two phases. First the clinical context was created from international guidelines, and systematic reviews on electronic fetal monitoring. Technical literature was then searched in the following order: IEEE Xplore, Pub Med, Science Direct and Google Scholar using combinations of the following search strings: cardiotocography, fetal heart rate, machine learning, deep learning, signal digitization, image segmentation and mobile health. Special consideration was given to using peer reviewed work that uses the open CTU-UHB or UCI datasets, since results on these can be directly compared to our own, and the original papers of the algorithms used in our implementation.

Sources were included if (a) they outlined criteria for interpreting a CTG, (b) they presented or assessed an automated method of analysis for the CTG, (c) they concerned the extraction of the curves from the images, or (d) they referred to the method used in the implementation. Articles from popular press and claims not subject to review were excluded.

2.2 Domain and theoretical foundations

2.2.1 CTG interpretation criteria

The IST consensus criteria from FIGO specify what these features are (see table 1). The clinician assesses the following parameters (see figure 1, which shows an example of a TRACE and the features discussed below) [2] ... The baseline: the average FHR in the absence of accelerations and decelerations in a 10-minute window with normal baseline between 110-160 bpm; Variability: a measure of the FHR fluctuations around the baseline; absent variability is ominous while reduced variability can be due to hypoxia or fetal sleep... Acceleration: an abrupt increase of more than 15 bpm of the FHR that lasts for more than 15 s, considered reassuring; Deceleration: a decrease of more than 15 bpm of the FHR that lasts for more than 15 s with early, variable, late, and prolonged dependent on the time (timing relative to the contractions) and depth and duration of the decelerations;.. Uterine contractions:

is characterized by the number of contractions in a 10-minute window and a contraction frequency of more than 5 contractions; Based on all of these parameters, the FIGO guidelines classify a trace as normal, suspicious or pathological while the NICE guideline organizes it into a three-tier classification of optimal, suboptimal and abnormal [4]. These three tiers correspond to the three possible labels (Normal, Suspect, Pathological) in the data set we use from UCI.

2.2.2 Clinical value and limitations of CTG

A Cochrane review reported that although continuous CTG reduced neonatal seizure, it did not reduce perinatal death or cerebral palsy and resulted in more caesareans and instrumental births [9]. One reason for the failure to make a difference was different interpretation. Computerized interpretation was predicted to improve management, but the large INFANT randomized trial found that decision support based on computerized interpretation of fetal heart rate did not improve neonatal outcome over that achieved with experts' visual interpretation [10]. This observation has relevance for this project: automation as support for improved access and consistency, not automation as a substitute for interpretation.

2.2.3 Technical foundations

2.3 Related research

Computerized CTG analysis. Sis Porto 2.0 was one of the first programs to extract FIGO features from digital CTG signals and generate alerts [3]. The 21 features computed by the programmed for 2126 antepartum recordings, with expert NSP labels, comprise the UCI Cardiotocography data [5]. Commercial systems such as used in the INFANT trial work on the same principles on the digital signal [10].

Tabular data ML Several papers have reported experiments with classifiers on the UCI data. Cömert and Kocandas investigated different machine-learning algorithms for FHR classification, finding ensemble and neural classifiers to outperform linear and base classifiers [19]. Spilka et al demonstrated that nonlinear features of the FHR can bring discriminative power to the traditional set of FIGO features [20]. Overall, the machine learning literature on this three-class task shows that ensembles of trees

perform well on this three-class classification task. The Suspect class is particularly difficult to distinguish.

Deep learning on CTG signals. The CTU-UHB database has 552 intrapartum recordings with umbilical-artery pH and other outcomes (distributed via Physio Net). Petrozzi Ello et al. used these signals to train multi-modal CNNs for detecting fetal compromise, while Zhao et al. used recurrence plots as inputs to a CNN to classify FHR signals. Georgieva et al. reviewed the field and concluded that outcome labels such as pH were loosely coupled to visual NSP categories.

Curve extraction Image curve extraction from scans or photos of chart plots is a classic digitization challenge. Conventional approaches generally perform a series of steps to remove the grid, convert the image to black-and-white, find the curve, then convert the pixel locations to real-world units. Deep learning techniques such as U-Net [15], which are commonly used for general thin-structure segmentation (for example, vessel segmentation in medical images), as well as object detection architectures such as YOLO [23], [24], which have been applied for chart detection in captured images. Nevertheless, there is little published evidence of the performance of these methods for CTG paper strips.

Table 2.1: Comparison of representative related work with this study

Work	**Input**	**Method**	**Output**	**Limitation for this problem**
SisPorto 2.0 [3]	Digital FHR/UC	Rule-based feature extraction	FIGO features, alerts	Needs monitor signal; no image input
Cömert & Kocamaz [19]	UCI features	Classical ML comparison	NSP class	Tabular only
Spilka et al. [20]	Digital FHR	Nonlinear features + classifier	Normal/abnormal	Signal only
Petrozziello et al. [7]	CTU-UHB / clinical signals	Multimodal CNN	Fetal compromise	Signal only; outcome labels
Zhao et al. [8]	CTU-UHB FHR	Recurrence plot + CNN	Hypoxia class	Signal only
This study	Phone photo of printout	Detection + trace segmentation + XGBoost	Signals, features, NSP class, quality flag	No window-level clinical labels for images

2.4 Existing systems products and techniques

Commercial pregnancy apps focus on tracking symptoms, reminders for appointments, weight and nutrition diaries and information. Of the frequently available applications we reviewed for the group proposal, not one provides an analysis of CTG traces. Hospitals with CTG monitors and central surveillance systems offer computerized interpretation but only at the bedside or on the hospital network, and only available to clinicians. Home fetal Dopplers allow parents to listen to the fontal heartbeat but do not interpret, possibly leading to false reassurance.

In terms of technique, the building blocks are mature: OpenCV for classical image processing [14], scikit-learn [25] and XGBoost [18] for classical classifiers, PyTorch [10] for compact segmentation networks [26], Fast API [27] for typed REST services with automatic documentation. The innovation in this component does not sit in the algorithms: it is the process of putting them together and experimenting with them, in the context of the task of reading a photographed CTG strip in a patient-facing workflow.

2.5 Gap synthesis and conceptual framework

The review demonstrates that the task has been divided into two halves, with the bridges seldom built between the two. Classification of CTG features on one side, mostly studied, with excellent results reported for clean digital data; curve digitisation, on the other side, a generic technique and not one used on CTG printouts, per se. Between these two extremes is one feature bridge: do the features based on the reconstructed signal agree with those based on the real signal, and if not, no good paper classification system, no matter how good, will produce reliable output on images.

The framework of this study as depicted in Figure 3.1 (chapter 3, next chapter) is, therefore, structured on these three levels: 'signal consistency' 'feature consistency' and 'classification accuracy' It also explicitly considers signal quality as a primary target output, so that classification can refuse to do so when its reconstruction is not of sufficient quality. This framework fills the gaps identified in Section 1.4.

3 RESEARCH METHODOLOGY

3.1 Research design

This project adhered to the principles of the Design Science Research (DSR) paradigm where knowledge is generated by building and evaluating an artefact to solve a relevant problem [28]. Peffers et al. proposed a process model of DSR consisting of six activities [29]: (1) problem identification based on literature review and clinical discussions, (2) determination of objectives (Section 1.5), (3) design and development of the CTG interpretation module, (4) demonstration (rendered CTG images and via the API), (5) evaluation based on the quantitative framework outlined in the current chapter, and (6) communication through this thesis and the group demonstrations.

Development was incremental. New extraction algorithms were developed as replaceable implementations, all behind a shared interface, ground-trothed, and only removed when superseded. For instance, the original Otsu-threshold extractor has been replaced with one adding grid-suppression and local adaptive thresholding, then connected-component trace segmentation, and more recently a learned U-Net, leaving four available. The work is experimental and quantitative, making direct, controlled comparisons between algorithms on the same samples.

3.2 Data participants or study materials

No human participants were recruited for the technical evaluation. Three data sources were used, summarised in Table 3.1.

Table 3.1: Datasets used in this study

Dataset	Content	Use in this study
UCI Cardiotocography [5]	2,126 antepartum records; 21 SisPorto features; expert NSP labels (1,655 Normal, 295 Suspect, 176 Pathological)	Training and cross-validated comparison of classifiers
CTU-UHB Intrapartum CTG Database [6]	552 intrapartum recordings with FHR and UC sampled at 4 Hz and clinical outcome metadata	Source of ground-truth signals, cut into 5-minute windows

Dataset	Content	Use in this study
Rendered paired image dataset (group data-generation pipeline, referred to in the code as 'Project 1')	CTG-style images rendered from CTU-UHB windows, with clean and augmented variants, graph bounding boxes and links to the source signal; train / validation / test splits	Training the U-Net and YOLO detector; evaluating detection, reconstruction and the feature bridge

Each rendered image is accompanied by a JSON label containing the image file name, the graph bounding box, the augmentation type and a reference to the source signal file, which holds the FHR and UC arrays and the sampling rate. A five-minute window at 4 Hz yields 1,200 samples per signal. Ground-truth FHR values outside 30–240 bpm, including the zeros that mark signal loss in CTU-UHB, are treated as missing when computing metrics.

The UCI class distribution is strongly imbalanced (77.8% Normal, 13.9% Suspect, 8.3% Pathological). The class prior stored in the trained XGBoost model (base scores of 1.322, −0.403 and −0.919 in log-odds) corresponds exactly to these proportions, confirming that the full dataset was used for the final fit.

A label audit script was written to check whether the rendered windows could be linked to NSP labels. It confirmed that neither the signal files nor the image labels contain an NSP field, and that the CTU-UHB metadata provides outcome measures such as umbilical pH and Apgar scores rather than visual NSP categories. This finding shaped the evaluation design described below.

3.3 Experimental or study procedure

The experiments were conducted in four stages, each implemented as a reproducible script.

3.3.1 Classifier comparison

The 21 UCI features were used as inputs and the NSP label as the target. Four candidate models were defined: logistic regression with feature standardisation (maximum 1,000 iterations); a random forest with 200 trees; an RBF-kernel support vector machine with standardisation and probability outputs; and XGBoost with 200 boosting rounds, maximum depth 6, learning rate 0.1 and the multi-class log-loss objective. All models were evaluated with stratified five-fold cross-validation

(shuffled, random state 42) using out-of-fold predictions, so that every record is predicted by a model that did not see it during training. The model with the highest macro-F1 was then refitted on all records and saved together with its label encoder.

3.3.2 Graph detection

The contour-based OpenCV detector was applied to clean rendered images and the predicted bounding box was compared with the ground-truth box using intersection over union (IoU), both overall and by augmentation type. A YOLOv8n detector [24] was trained on the same boxes converted to YOLO format (50 epochs, image size 640, early-stopping patience 10) as an optional alternative.

3.3.3 Signal reconstruction and U-Net training

For each sample, the ground-truth graph crop was split into FHR and UC panels and passed to every extractor. Reconstructed signals were compared with the ground-truth signals sample by sample. The TraceUNet was trained on 150 clean training panels, of which 128 were used for training and 22 for validation after removing panels with empty masks. Training masks were generated by drawing the ground-truth FHR signal into panel coordinates with a three-pixel line, so that the network learns the true trace position rather than the output of a threshold. Panels and masks were resized to 384 × 120 pixels. The network was trained for 12 epochs with the Adam optimiser [30] (learning rate 10^{-3}, batch size 8), a combined BCE and Dice loss, and a learning-rate reduction on plateau; the weights with the lowest validation loss (0.9496) were retained.

3.3.4 Feature bridge and end-to-end runs

For up to 200 samples per configuration, CTG features were computed twice, once from the ground-truth signals and once from the reconstructed signals, using identical code, and the absolute differences were recorded. The full analyser was also run on sample images to verify the complete path from photograph to classification output.

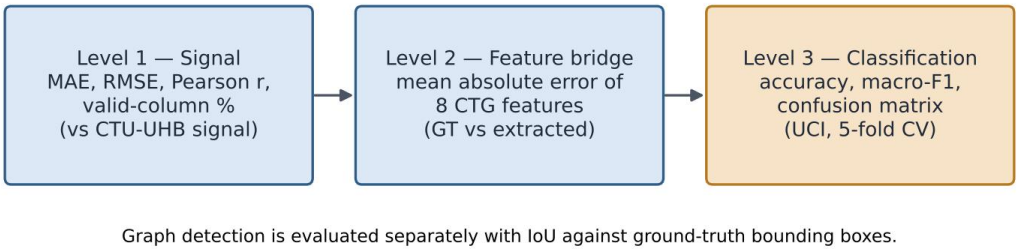


Figure 3.1: Three-level evaluation framework used in this study

3.4 Measures and evaluation metrics

Table 3.2 lists the metrics used at each level of the framework in Figure 3.1.

Table 3.2: Evaluation metrics

Level	Metric	Definition / purpose
Detection	IoU	Area of overlap divided by area of union between predicted and ground-truth graph boxes; IoU ≥ 0.5 counted as correct
Signal	MAE, RMSE	Mean absolute and root-mean-square difference (bpm for FHR, units for UC) over valid ground-truth samples
Signal	Pearson r	Correlation between reconstructed and true signals, measuring shape agreement independent of offset
Signal	Valid-column %	Share of time samples for which a trace pixel was found before interpolation; also drives the quality gate
Feature	Mean feature error	Mean absolute difference across eight features: baseline, STV, LTV, accelerations, decelerations, mean UC, max UC, contraction count
Classification	Accuracy, macro-F1	Macro-F1 averages per-class F1 so that the minority Suspect and Pathological classes weigh equally with Normal
Classification	Confusion matrix	Shows which classes are confused, particularly Suspect versus Normal

Macro-F1 was chosen as the primary classification metric because accuracy on the imbalanced UCI dataset can exceed 77% by predicting Normal for every record. The mean feature error was chosen as the primary criterion for selecting the production extractor because the classifier consumes features, not raw signals.

3.5 Data analysis

For each extractor and configuration, we took the per-sample metrics, averaged across samples (standard deviations for MAE), and output them to JSON reports in our report's directory in the project, so that they could be redone for each revision in the future. Classifier results were summarized by accuracy, macro-F1, per-class precision and recall and out-of-fold confusion matrices. Feature importance for the production XGBoost model was calculated by adding the reduction in loss (gain) for each split on that feature over all 600 trees and normalizing to percentage. Since all extractor comparisons were on the same set of samples, the performance difference is to methods rather than the nature of the input.

3.6 Ethics privacy security and responsible AI

Both datasets are open-access and fully de-identified; CTU-UHB is shared via Physio Net under an open license [21], [6]. We did not gather identifiable patient data during development. In deployment, CTG images and results are personal health data, and the system architecture complies with the recommendations of the (UK) Personal Data Protection Act, No. 9 of 2022 [31]; this includes minimization (the AI service is stateless and does not retain images), purpose limitation, transport encryption (HTTPS/TLS) and role-based access through the backend.

Responsible-AI considerations incorporated from the beginning, not 'bolted on'. The service has never shown a classification as a diagnosis, all results go through the doctor review interface, and if the quality of reconstruction is poor, we do not show a classification but explain to the user what the system did do. The API and evaluation reports say clearly that the 0.917 macro-F1 is on the tabular UCI features and is not an image-classification accuracy score. The label audit says clearly that labels may not be based on model predictions.

3.7 Reproducibility and research artefacts

All experiments are scripted and use fixed random seeds (42) for data splits, model initialisation and shuffling. Paths to datasets and output directories are kept in a configuration file (*configs/paths.json*) and pipeline choices in *configs/pipeline.json*,

so that the same scripts run on different machines. The research artefacts are listed in Table 3.3.

Table 3.3: Research artefacts

Artefact	Location	Description
Source code	ctg-ai-service/src	Detection, extraction, features, classification, pipeline, API
Experiment scripts	ctg-ai-service/scripts	Training, evaluation, comparison and audit scripts
Trained classifier	models/classifier/classifier.pkl	XGBoost model, feature list and label encoder (1.1 MB)
Trained U-Net	models/fhr_unet/trace_unet.pt	TraceUNet weights (1.9 MB) and training_meta.json
Evaluation reports	reports/*.json	Classifier comparison, extractor comparisons, feature bridge, label audit
Test suite	tests/test_ctg_pipeline.py	Unit and synthetic round-trip tests (Section 5.4)

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

Requirements were derived from the group proposal, discussions with clinicians and the technical constraints identified in the literature. Table 4.1 lists the functional requirements for which this component is responsible, and Table 4.2 the non-functional requirements.

Table 4.1: Functional requirements of the CTG interpretation module

ID	Requirement	Priority
FR1	Accept a CTG photograph (JPEG/PNG) through an HTTP endpoint and reject non-image uploads	Must
FR2	Pre-process the photograph (resizing, denoising) and localise the CTG graph region	Must
FR3	Reconstruct FHR and UC time series at 4 Hz from the graph region	Must
FR4	Compute baseline, STV, LTV, accelerations, decelerations, mean and maximum UC and contraction count	Must
FR5	Classify the record as Normal, Suspect or Pathological with class probabilities	Must
FR6	Report signal quality and withhold classification when quality is insufficient	Must
FR7	Accept manually entered CTG values in the same feature schema	Should
FR8	Report service health and the active model configuration	Should
FR9	Allow extraction methods and detectors to be switched by configuration without code changes	Could

Table 4.2: Non-functional requirements

Category	Requirement
Performance	Return an analysis within 5–10 s of upload on server hardware
Reliability	Fail gracefully with a structured error when no graph is detected or the image cannot be decoded
Safety	Never present an unreliable classification; state the limits of tabular accuracy in model metadata
Security and privacy	Stateless processing; HTTPS/TLS in transit; no persistent storage of images in the AI service
Maintainability	Modular packages with common interfaces for each stage; configuration-driven pipeline

Category	Requirement
Portability	Run on Windows and Linux; avoid fragile native dependencies (pure-NumPy peak detection and filtering)
Scalability	Stateless service suitable for horizontal scaling behind a load balancer

4.2 Component architecture within the complete solution

Within the overall system (Figure 1.1), the mobile application uploads the photograph to the backend, which forwards it to the AI CTG Interpretation Service. The service returns a JSON analysis that the backend stores with the patient record and places in the doctor's review queue. After review, the doctor's comments and the approved result are returned to the mother. The AI service has no direct knowledge of users or sessions, which keeps it simple, testable and replaceable.

Internally, the component is a linear pipeline of interchangeable stages, shown in Figure 4.1. Each stage is a Python package with a common function signature, and the analyser assembles the stages selected in the configuration.

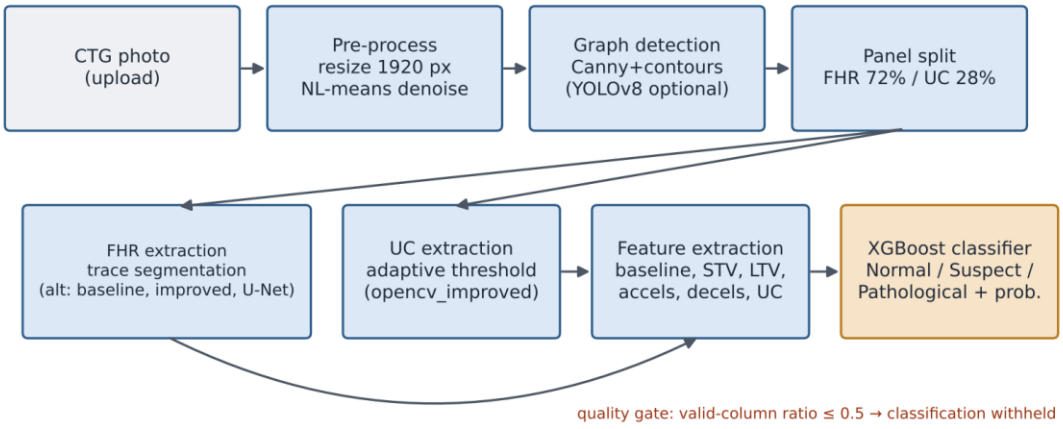


Figure 4.1: Internal processing pipeline of the CTG interpretation module

- **graph_detection** – OpenCV contour detector and optional YOLOv8 detector; both return the same dictionary (detected, bbox, confidence, method).
- **fhr_extraction** and **uc_extraction** – registries (*FHR_EXTRACTORS*, *UC_EXTRACTORS*) mapping method names to functions that return the signal, valid ratio, quality and method name.

- **extraction.trace_utils** – shared panel splitting, grid suppression, binarisation, component selection and pixel-to-value mapping.
- **feature_extraction** – computes the CTG feature dictionary from any FHR/UC pair.
- **classification** – UCI loader, model comparison and the *CTGClassifier* wrapper that maps features to the model input.
- **pipeline** – pre-processing and the *CTGAnalyzer* orchestrator; **api** – FastAPI application; **evaluation** – dataset loader, signal metrics and feature bridge.

4.3 Design alternatives and decisions

Several design alternatives were considered and evaluated. Table 4.3 summarises the main decisions and their rationale.

Table 4.3: Design alternatives and decisions

Decision	Alternatives considered	Choice and rationale
Graph localisation	OpenCV contours; YOLOv8 detector	OpenCV by default (no extra weights, fast, adequate on rendered data); YOLOv8 used automatically when trained weights exist, with OpenCV as fallback
FHR reconstruction	Otsu baseline; improved adaptive threshold; trace segmentation; U-Net	Trace segmentation in production: lowest feature-bridge error (6.56 vs 9.18 for U-Net) despite U-Net's better signal agreement
UC reconstruction	Otsu baseline; improved; trace segmentation	Improved adaptive-threshold extractor as default
Classifier	Logistic regression; random forest; SVM; XGBoost	XGBoost: highest cross-validated macro-F1 (0.917)
Training masks for U-Net	Threshold the image; draw from ground-truth signal	Draw from signal, so the network learns the true trace rather than reproducing threshold errors
Low-quality handling	Always classify; classify with warning; withhold	Withhold when valid-column ratio ≤ 0.5 for FHR or UC, with an explanatory note
Service framework	Flask; FastAPI	FastAPI: typed request handling, asynchronous uploads and automatic OpenAPI documentation
Peak detection	SciPy; custom NumPy	Custom NumPy implementation to avoid native-library failures on locked-down Windows machines

The most significant decision concerned FHR extraction. The U-Net achieved lower signal error and higher correlation than trace segmentation, which would normally

favour it. However, the classifier never sees the raw signal; it sees features such as the number of accelerations. Because trace segmentation produced features closer to the ground truth, it was chosen for production. This decision directly reflects the hypothesis stated for RQ3.

4.4 Detailed design

4.4.1 Pre-processing and graph detection

Uploaded images are scaled to width 1920 px while maintaining aspect ratio and denoised using OpenCV colour non-local-means [13] filter. The OpenCV detector converts to grey, applies a 5 × 5 Gaussian kernel and Canny edge detection (thresholds 30 and 100), closes contours using a morphological operation, then determines external contours. Candidates are rejected if their bounding rectangle overlaps less than 15% of the image and aspect ratio is outside the range [1.2, 5.0]. The percentage of the image area covered by the candidate rectangle is scaled to a 0.6 score plus a centrality score scaled to 0.4, and the largest candidate wins. In the absence of a valid contour, the central 90% of the image is used with a 0.3 confidence level, so that quality validation filters, rather than detection, determine if the image content is acceptable.

4.4.2 Trace reconstruction

The graph crop is split at 72% of its height into the FHR and UC panels. Each panel is enhanced with CLAHE (clip limit 2.0, 8 × 8 tiles) [12]; horizontal and vertical grid structures are suppressed using morphological opening with 40 × 1 and 1 × 20 kernels; and the result is smoothed and binarised with a Gaussian adaptive threshold (block size 21, constant 5), followed by morphological opening and closing. In the trace-segmentation method, connected components are then filtered (width ≥ 25% of the panel, height ≤ 85%, area ≥ 50 pixels) and the component maximising (relative width × area) is kept as the trace.

For every pixel column, the median row of trace pixels is taken as the trace centre; columns with no trace pixels, or with trace pixels in more than 40% of the rows (indicating a vertical grid artefact), are skipped. Column positions are mapped to 1,200 time samples, and row positions to physical values by v = v_max − (y / (h −

1)) × (v_max − v_min), with 30–240 bpm for FHR and 0–100 for UC. Missing samples are filled by linear interpolation and the signal is smoothed with a five-point median filter. The fraction of samples that were directly observed is reported as the valid ratio.

4.4.3 TraceUNet

The learned alternative is a compact three-level U-Net (Figure 4.2). Each encoder and decoder block applies two 3 × 3 convolutions with batch normalisation and ReLU; down-sampling uses 2 × 2 max-pooling and up-sampling uses bilinear interpolation, with skip connections concatenating encoder features into the decoder. A final 1 × 1 convolution and sigmoid produce a trace-probability map, which is thresholded at 0.5, resized to the panel size and converted to a signal using the same column-centre procedure as the classical methods. With 32, 64 and 128 channels the network has 472,257 trainable parameters and a weight file of 1.9 MB, small enough for CPU inference.

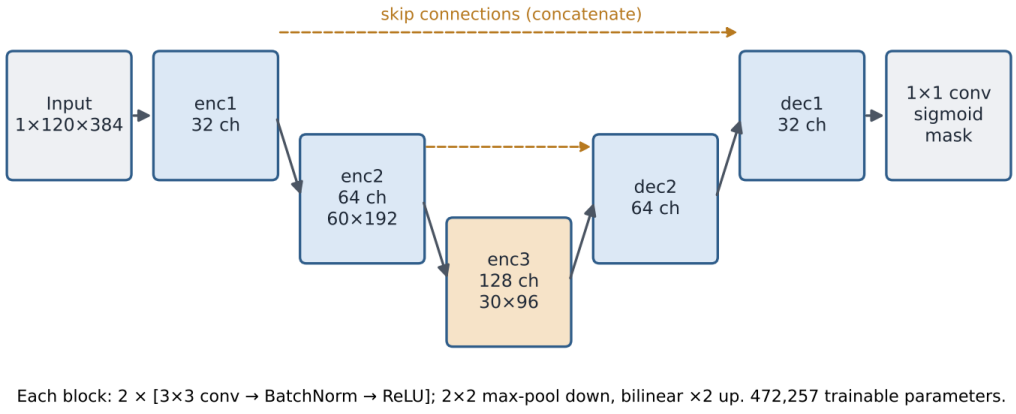


Figure 4.2: Architecture of the TraceUNet FHR segmentation network

4.4.4 Feature extraction

Features are computed from the reconstructed 4 Hz signals as follows. The **baseline** is estimated as the 25th percentile of valid FHR samples, which is robust to accelerations. **STV** is the mean absolute difference between consecutive samples and **LTV** the mean standard deviation over one-minute windows. An **acceleration** is counted for every run in which the FHR exceeds the baseline by at least 15 bpm for at least 15 s (60 samples), and a **deceleration** for every run at least 15 bpm below the

baseline for at least 15 s, following the FIGO thresholds [2]. For UC, the mean and maximum values are reported and contractions are counted as peaks above the 60th percentile separated by at least 30 s.

4.4.5 Classification and response schema

The classifier inputs 21 UCI features. Reconstructed features are reintegrated on to this scheme (baseline onto LB, Mean and Median; STV onto ASTV and MSTV; LTV onto ALTV, MLTV and Variance; accelerations onto AC) and UCI features that have not yet been reconstructed from an image (for example FM - the histogram descriptors and the deceleration sub-types of DL, DS and DP) are filled with typical values. The classifier outputs the predicted status, the maximum class probability (as confidence) and the probability distribution over all classes. The analyzer merges detection, extraction, feature and classification outputs into a single JSON response which records the pipelines configuration. Thus, each result can be traced back to the methods which generated it.

.4.5 Standards and professional practice

Definitions of clinical features follow the FIGO consensus statement [2] and are consistent with NICE guidance [4]. Although the academic prototype is not a regulated device, the development of it was informed by the life-cycle processes of medical-device software (IEC 62304 [32]) and risk management (ISO 14971 [33]), where the main hazards (misdetection of the graph, mis reconstruction of the trace and misclassification) were addressed by quality gating, clinician review and explicit constraints on model capabilities; software quality attributes were informed by ISO/IEC 25010 [34]. Professional practices included code and configuration file version control rather than hard-coded paths, fixed random seeds, type hints across the entire codebase and documented evaluation reports.

5 IMPLEMENTATION INTEGRATION AND TESTING

5.1 Development environment and technology stack

The component was developed in Python on Windows, with care taken to keep it portable to Linux servers. Table 5.1 lists the main technologies.

Table 5.1: Technology stack

Layer	Technology	Purpose
Language	Python 3.10+	Type-hinted implementation of all stages
Image processing	OpenCV, NumPy	Pre-processing, detection, thresholding, morphology, connected components
Deep learning	PyTorch [26]	TraceUNet training and inference
Object detection	Ultralytics YOLOv8 [24]	Optional learned graph detector
Machine learning	scikit-learn 1.9 [25], XGBoost [18], pandas	Data loading, cross-validation, classifiers
Service	FastAPI [27], Uvicorn	REST API for analysis
Clients (other components)	Flutter mobile app; doctor web interface; cloud database	Upload, review and storage

5.2 Component implementation

The code base contains about 3,000 lines of Python across nine packages and fourteen scripts. Key implementation details are described below.

5.2.1 Configuration-driven analyser

The *CTGAnalyzer* class is created from configuration through *CTGAnalyzer.from_config()*, which loads the classifier if *models/classifier/classifier.pkl* exists, enables the YOLO detector if *models/graph_detector/best.pt* exists, and reads the selected extractors from *configs/pipeline.json*, defaulting to the OpenCV detector, trace-segmentation FHR extraction and improved UC extraction. If the U-Net is selected but its weights or PyTorch are missing, the analyser falls back to a classical extractor and records the fallback in the method name. This means the same code runs on a minimal server and on a development machine with every model installed.

5.2.2 Quality gating

Each extractor reports a valid ratio. If either the FHR or the UC valid ratio is 0.5 or lower, the overall signal quality is set to *poor*, the analysis status to *low_confidence*, and classification is skipped with the note 'signal extraction quality too low for reliable analysis'. If no graph is detected, the response has *success* = *false* and *analysis_status* = *failed*. These states allow the mobile application to ask the mother to retake the photograph rather than showing a misleading result.

5.2.3 Classifier training and persistence

The *train_classifier.py* script loads the UCI *Raw Data* sheet, removes incomplete rows, runs the cross-validated comparison, refits the best model on all records and saves a dictionary containing the model, the feature-column list and the label encoder. The persisted production model is an XGBoost classifier with the *multi:softprob* objective, 200 rounds, maximum depth 6 and learning rate 0.1. Because a separate tree is grown for each class in each round, the model contains 600 trees.

5.2.4 Service interface

The FastAPI application exposes three endpoints (Table 5.2). The analyser is created lazily on first use and reused for subsequent requests, so models are loaded only once.

Table 5.2: Service endpoints

Method and path	Input	Output
GET /health	–	Service status and whether the classifier and YOLO detector are loaded
GET /models/info	–	Active detector, extractors and classifier, plus a note that the 0.917 F1 is tabular-only
POST /analyze	Multipart image file	JSON: success, analysis_status, graph_bbox, detection method and confidence, signal_quality, FHR/UC extraction method and valid ratio, features, classification, pipeline_config

5.3 Integration with the other components

Integration with the rest of the Pregnancy Support System is through the */analyze* endpoint. The mobile application captures or selects the photograph and uploads it to

the backend; the backend calls the AI service, stores the JSON response with the patient record and creates a review task in the doctor interface. The doctor sees the detected graph region, the reconstructed traces, the feature values, the classification and its probabilities, and can accept or amend the interpretation and add comments before the result is released to the mother. A *low_confidence* or *failed* response is surfaced to the mother as a request to retake the photograph with guidance on lighting and framing.

Keeping the AI service stateless and contract-based allowed it to be developed and tested independently of the other components. The JSON response schema forms the contract with the components developed by the teammates responsible for the mobile application and the doctor interface, and the *pipeline_config* and *method* fields allow the review interface to show which methods produced each result. [Describe here the integration tests carried out with your teammates' components (e.g., mobile upload → backend → AI service → doctor dashboard) and their outcome.]

5.4 Verification and software testing

Verification was carried out at three levels. First, each stage was checked against ground truth using the evaluation scripts described in Chapter 3. Second, the complete path was exercised with *run_end_to_end.py*, which runs the analyser on test images and records the outputs. Third, a unit and synthetic round-trip test suite (*tests/test_ctg_pipeline.py*, written with Python's standard *unittest* module so that it has no extra dependencies) was created to check the behaviour of individual functions and to stress the extractors with images of known content.

The round-trip tests render a known FHR signal (baseline 140 bpm with sinusoidal variability, two 20 bpm accelerations of 25 s and one 25 bpm deceleration of 22 s) and a UC signal with three contractions onto a CTG-style strip with a pink grid, a panel divider and a frame, then run detection, extraction and feature computation and compare the results with the known values. Five random seeds were used, with and without additive Gaussian noise ($\sigma = 12$ grey levels). Table 5.3 summarises the tests.

Table 5.3: Unit and round-trip test results

Test group	What is checked	Result

Test group	What is checked	Result
Signal utilities (5 tests)	Pixel-to-value mapping, peak spacing, median filtering, gap interpolation, panel split	All passed
Feature extraction (7 tests)	Baseline on flat and partly missing traces, zero variability, acceleration duration rule, deceleration, contraction count, all features on synthetic ground truth	All passed
Graph detection (1 test)	Detection of the graph region on a rendered strip	Passed; IoU = 0.996
Round trip, OpenCV baseline	FHR MAE on clean / noisy strips	Passed; 0.92 / 0.93 bpm
Round trip, trace segmentation	FHR MAE on clean strips (known issue)	Expected failure: 1.75–3.76 bpm on 3 seeds, ≈ 98 bpm on 2 seeds
Classifier (1 test)	Valid class and probabilities summing to 1	Skipped in the test environment (XGBoost not installed); [re-run locally and record]

The round-trip tests exposed a failure mode of the trace-segmentation method. On some strips the adaptive threshold fragments the trace at grid intersections, and the component-selection score (relative width × area) then favours a full-width horizontal ruling line, such as the top border of the frame, over the largest trace fragment. On a successful seed the trace scored about 5,826 against about 4,839 for the border, a narrow margin; on the failing seeds the border won and the reconstructed FHR became a flat line at about 240 bpm (Figure 5.1). The OpenCV baseline extractor, which uses the median of all dark pixels, performed best on these idealised renderings because their background is uniform. This result does not contradict the selection of trace segmentation on the rendered CTU-UHB data, but it shows that the method needs a guard against near-straight full-width components, as discussed in Chapter 7.

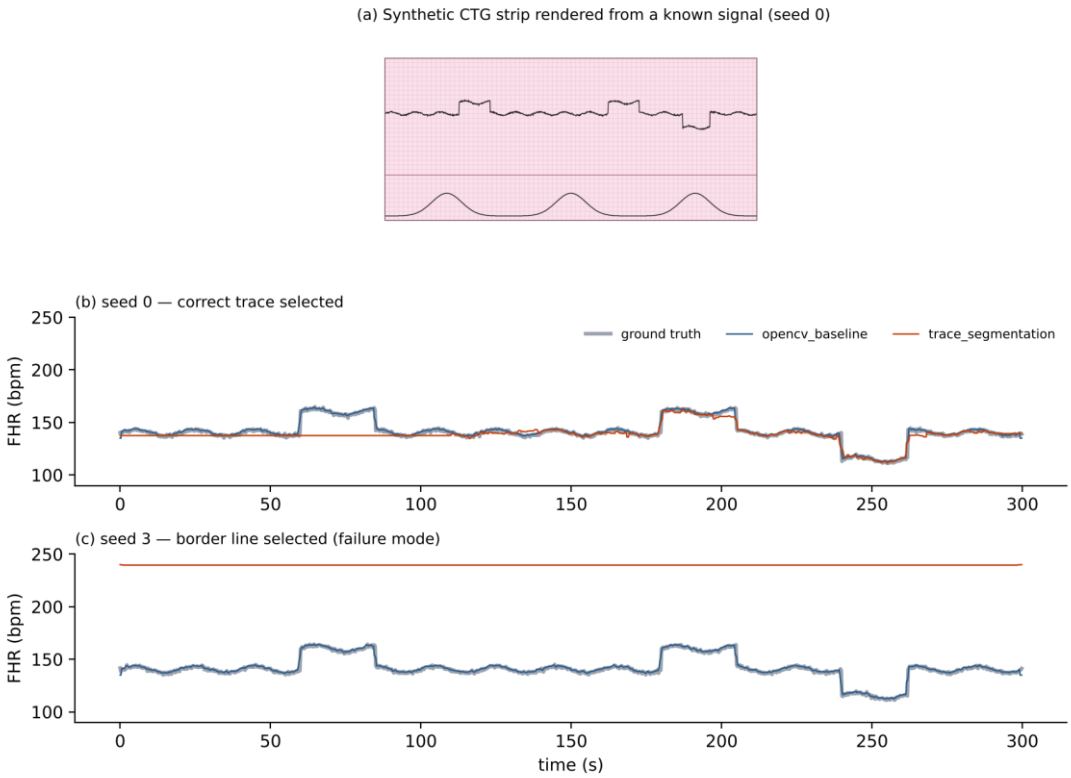


Figure 5.1: Synthetic round-trip test: (a) rendered strip; (b) correct reconstruction; (c) failure in which a border line is selected as the trace

5.5 Deployment and operation

To run the service, invoke scripts/runserver.py, which runs Unicorn with port 8000. The intention is to run this service in production in a container that is connected behind the group backend, with HTTPS in front of the gateway. Because it is stateless, we can add additional services to the back end behind a load balancer as necessary. Health checks of the container are supported with /health, and operators and the doctor interface can check which models are enabled with /models/info. Model files are versioned with their training metadata (e.g., trainingmeta. json in the case of the U-Net) so that a deployed output can always be traced back to the model that generated it. [Add actual hosting environment (e.g., cloud provider, VM size) and measured response time per image, if available.]

6 RESULTS AND VALIDATION

6.1 Research results

6.1.1 Classifier comparison (RQ1)

Table 6.1 shows the cross-validated performance of the four classifiers on the UCI dataset. XGBoost achieved the highest macro-F1 of 0.917 and was selected as the production classifier.

Table 6.1: Five-fold cross-validated classifier performance on UCI tabular features (n = 2,126)

Model	Accuracy	Macro-F1
Logistic regression	[from reports/classifier_comparison.json]	[...]
Random forest (200 trees)	[...]	[...]
SVM (RBF)	[...]	[...]
XGBoost (selected)	[...]	**0.917**

Figure 6.1 shows the gain-based feature importance of the production model. The three variability-related and level features ASTV (22.9%), ALTV (18.5%) and Mean FHR (15.7%) account for 57% of the total gain, followed by accelerations (8.9%) and mean short-term variability (7.5%). Two features, DS (severe decelerations) and DR (repetitive decelerations), were never used for splitting, which reflects how rarely they are non-zero in the dataset.

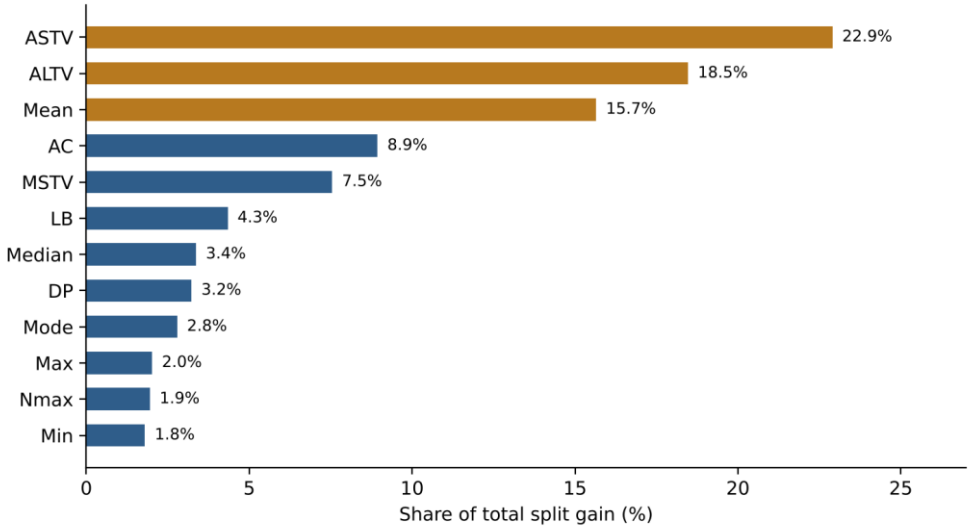


Figure 6.1: Gain-based feature importance of the production XGBoost classifier (top 12 of 21 features)

[Insert the confusion matrix for XGBoost from reports/classifier_comparison.json and comment on Suspect-class recall.]

6.1.2 Signal reconstruction and the feature bridge (RQ2, RQ3)

Table 6.2 compares the two leading FHR extractors on the rendered CTU-UHB data. The U-Net achieved better signal-level agreement (lower MAE and higher correlation) than trace segmentation, but trace segmentation produced features much closer to those of the ground-truth signal, with a mean feature error of 6.56 against 9.18 for the U-Net, a relative reduction of 29% (Figure 6.2).

Table 6.2: Comparison of FHR extractors on rendered CTU-UHB samples

Extractor	FHR MAE (bpm)	Pearson r	Mean feature error
OpenCV baseline (Otsu)	[...]	[...]	[...]
OpenCV improved	[...]	[...]	[...]
Trace segmentation (production)	[...]	[...]	**6.56**
U-Net	[lower than trace segmentation]	[higher]	9.18

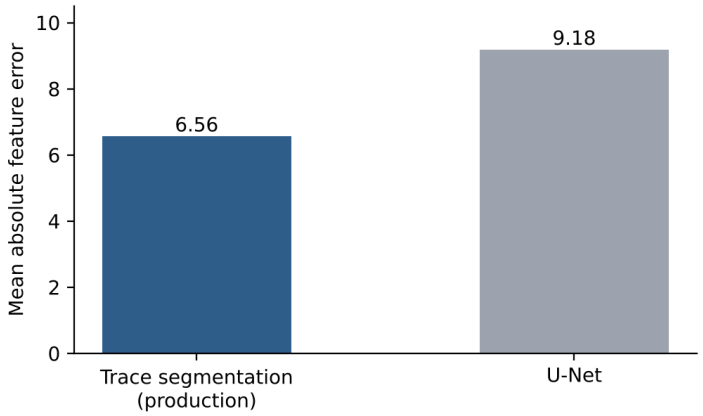


Figure 6.2: Mean absolute feature error for the two leading FHR extractors

These results answer RQ3: better signal agreement did not translate into better feature recovery. Signal MAE averages error over all samples, so a method that is slightly more accurate everywhere can still lose short, clinically important events such as the onset of a deceleration, or add small spurious excursions that the feature rules count as events.

6.1.3 Failure conditions (RQ4)

Two of the failures detected by the synthetic stress tests (Section 5.4) expose a single pattern of failure in trace segmentation: selection of the full-width ruler line on a more or less segmented strip: 2 out of 5 on clean synthetic strips, 3 out of 5 on noisy synthetic strips. For these, we still get valid-column ratio≥0.5, so this pass is missed by the quality gate. This is related to RQ4: valid-column coverage says only that there is some trace.

6.2 Product and technical results

The product delivered by this component is a working CTG analysis service with the following measured or verified properties:

- **Graph detection** – the OpenCV detector located the graph on rendered strips with IoU = 0.996 in the synthetic test. [Add the mean IoU and IoU ≥ 0.5 rate from reports/graph_detection_eval.json, overall and by augmentation type.]
- **U-Net training** – the TraceUNet converged to a best validation loss (BCE + Dice) of 0.9496 on 22 validation panels after training on 128 panels for 12 epochs.

- **Model footprint** – the classifier file is 1.1 MB and the U-Net weights 1.9 MB, so all models can be loaded on a CPU-only server.
- **Quality gating and error handling** – non-image uploads are rejected with HTTP 400, undecodable images with a clear message, failed detections with *analysis_status = failed*, and low-coverage reconstructions with *low_confidence* and no classification.
- **Configurability** – detectors and extractors can be changed through configuration, and the method actually used is recorded in every response.
- **Response time** – [record the measured average time per /analyze request] against the 5–10 s requirement.

6.3 User or stakeholder validation

The design was informed by practical knowledge shared by medical professionals during the proposal stage about CTG usage, maternal-care workflows and clinical considerations. This input shaped three design decisions: the choice of guideline-based features, mandatory doctor review before results reach the mother, and explicit feedback when an image is unsuitable. The three-tier output was aligned with the categories clinicians already use under the FIGO and NICE guidelines.

[If you conducted usability testing or expert review (e.g., a doctor comparing system output with their own reading for a set of CTG photographs, or mothers completing an upload task), report the participants, procedure and results here. If not, state clearly that formal stakeholder validation is planned future work.]

6.4 Integration results

The service was exercised through its HTTP interface with sample CTG images, and the returned JSON followed the documented response schema used by the mobile and doctor-interface components. The end-to-end script confirmed that the complete chain from image to classification runs without manual intervention and that failure states are returned in the documented form. [Report the results of the integrated system test with the mobile app, backend and doctor dashboard, including any issues found and fixed.]

6.5 Summary of findings

Table 6.3: Summary of findings by research question

Question	Finding
RQ1	XGBoost gave the best macro-F1 (0.917) on UCI tabular features; ASTV, ALTV and Mean FHR were the most influential features
RQ2	FHR and UC can be reconstructed from rendered CTG images; the U-Net gave the best signal agreement
RQ3	Better signal agreement did not mean better features: trace segmentation had a 29% lower feature error (6.56 vs 9.18) and was selected
RQ4	Trace segmentation can lock onto full-width ruling lines; the current coverage-based quality gate does not detect this, so a geometric guard is needed

7 DISCUSSION

7.1 Interpretation of findings

The key conclusion from this experiment is that the appropriate choice of reconstruction technique depends on the input of your downstream model. A U-Net trained on 128 panels learned to exactly position the trace at least on average, but the resulting features were worse than with a well-designed classical method. One is that the limited training set constrains generalisability over trace shape, especially for rare but information-rich events like deep decelerations. Another is that minor errors with a binary thresholder probability map could lead to small gaps or spurs which the event-counting rule counts as additional accelerations or decelerations.

Aside from preferences, the feature-importance analysis offers a further piece of evidence to suggest caution. The most important UCI features ASTV and ALTV are described in terms of the percentage of time for short- or long-term abnormal variability. The present mapping supplies the image-derived LTV and STV values (measured in bpm). The classifier is asking the image-derived measures to be on a different scale than those it was trained on - exactly the features it considers most. This, with the fixed set of default values for the deceleration sub-types, accounts for the macro-F1 of 0.917 not to transfer to the image input, explaining why this is given as a tabular result only.

7.2 Comparison with literature baselines and existing solutions

The table output is in line with those found in the literature for the UCI dataset: tree ensembles tend to outperform linear models and the Suspect class is the most difficult to recognize [19]. The dominating features are in line with the literature in which low variability features are indicative of a compromised patient [2] and with studies reporting the importance of variability descriptors [20].

This component, which reaches back to an earlier and less-proven signal-based deep-learning component [7], [8], is thus complementary to [3], as it involves the earlier step of extracting signals from paper, rather than a competition. Compared with Sis Porto and hospital systems [3], it does not need access to the monitor and can be used

by the mother (carried out on a phone). Unlike all known consumer pregnancy apps, it offers structured interpretation of a clinician reviewed CTG analysis. The negative results of the INFANT trial [10] are a reminder that even an accurate interpretation alone does not improve outcomes, hence the rationale for always keeping a doctor in the loop.

7.3 Research contribution

Our study makes three research contributions. First, it provides the first evidence that the use of signal-level metrics is not an adequate criterion to select a CTG reconstruction method, which can be replaced by a feature-level bridge metric. Second, the three-level evaluation framework suggested by our experiments enables to separate tabular classification accuracy from image-based reconstruction accuracy, which is a frequent source of over-claiming. Third, it establishes a specific failure mode for component-based trace selection and demonstrates that coverage-based quality measures are unaware of it, which motivates geometry plausibility measures.

7.4 Product contribution

The product contribution is a modular, documented and tested CTG analysis service that the Pregnancy Support System can call through a simple API. Its design choices (configurable stages, stateless operation, explicit quality states, traceable model metadata and small model files) make it practical to deploy and maintain, and allow improved models to be introduced without changing the mobile or web clients.

7.5 Limitations and threats to validity

- **Construct validity** – no window-level NSP labels exist for the rendered CTU-UHB images, so end-to-end classification accuracy from images could not be measured. CTU-UHB outcome measures (pH, Apgar) are not equivalent to visual NSP categories [22].
- **Feature-schema mismatch** – several UCI features are mapped approximately (ASTV and ALTV from STV and LTV in bpm) or filled with defaults (DL, DS, DP, FM and histogram features), as discussed in Section 7.1.

- **External validity** – reconstruction was evaluated on rendered images. Real phone photographs introduce perspective distortion, curved paper, glare, shadows, faded thermal print, different printer scales and annotations that the rendered augmentations may not fully represent.
- **Internal validity** – extractor comparisons used the training split with up to 200 samples; using the held-out test split would give a less optimistic estimate. The U-Net was trained on a small set (128 panels, 12 epochs) and may be under-trained relative to its capacity.
- **Fixed layout assumptions** – the 72/28 panel split, the axis ranges and the 300 s duration are fixed; different monitor models use different scales (for example 1 or 3 cm/min paper speeds), which would shift the time axis.
- **Clinical scope** – the feature rules do not yet classify deceleration types (early, late, variable, prolonged) or relate them to contractions, which FIGO considers essential.

7.6 Implications

For practice, the results suggest that a phone-based CTG reader can provide useful structured information for clinician review, provided it is honest about uncertainty and declines to classify unreliable images. For developers of similar systems, the main implication is to evaluate reconstruction at the level the downstream model uses and to avoid reporting tabular accuracy as system accuracy. For research, the work highlights the need for a public dataset of real CTG photographs paired with digital signals and expert annotations, which would allow the end-to-end question to be answered properly.

8 COMMERCIALISATION AND SUSTAINABILITY

8.1 Value proposition and target users

Primary users are pregnant women, particularly second and third trimester, who need scheduled checkups of their babies, but are geographically, financially or temporally constrained from attending hospitals. Secondary users are obstetricians, maternity clinics, private hospitals and telemedicine providers. The benefit to mothers is instantaneous, doctor validated information as currently obtained from a CTG print out, which the mother will already have, and a systematic record of her CTG results. The benefit for doctors is pre-processed, standardized information for rapid remote review and alerts to potential traces for urgent interpretation.

8.2 Market and competitive position

Conventional pregnancy apps are educational and progress charts only with no CTG interpretation capability, and computerised CTG analysis is available only via hospital monitors and central stations. The new module fills a void in between the two: it provides computerised, image-based CTG interpretation via the patient's smartphone and is then verified by a clinician. The key competitive edge is image-based input, with no extra hardware, and doctor-verified results, quality feedback and a longitudinal record of CTGs within an integrated pregnancy app.

8.3 Business or sustainability model

The group proposal outlines a mixed revenue model: subscription plans for mothers who want unlimited CTG analyses and history; clinic integration packages in which hospitals and obstetricians pay a licence fee for the doctor review dashboard; and a pay-per-scan option for occasional users. A free tier for basic use would support access for low-income users, with clinic licences providing the main recurring revenue. Table 8.1 summarises the estimated costs for this component from the proposal.

Table 8.1: Estimated development cost for the component (LKR)

Item	Estimated cost	Justification

Item	Estimated cost	Justification
Mobile device for testing	15,000	Camera-capture testing on real hardware
Software licences and tools	0 – 5,000	Mostly open-source tools
Cloud services (database and hosting)	3,000 – 5,000 per month	Model hosting, storage, authentication
Model training compute	5,000 – 10,000	GPU time for U-Net and detector training
Human effort (development and documentation)	30,000 – 40,000	Development, integration, testing and writing
Medical expert consultation	10,000 – 15,000	Validation of interpretation and risk levels
Miscellaneous	2,000 – 3,000	Internet, printing, transport

8.4 Scalability intellectual property and compliance

The stateless service architecture scales horizontally, and the small CPU-friendly models keep per-analysis costs low. The image-processing pipeline, the feature-bridge selection procedure and the combined workflow of mother upload, AI analysis and doctor verification may qualify for software copyright and, potentially, protection as a proprietary process. Open-source dependencies are used under permissive licences, and the CTU-UHB and UCI datasets are used under their respective terms; any commercial use of models trained on them must respect those terms.

Before commercial deployment, the product would need to comply with Sri Lanka's Personal Data Protection Act [31], obtain any approvals required by the national medicines and medical-device regulator for software that supports clinical decisions, and follow medical-device software and risk-management standards [32], [33]. Clinical validation on real CTG photographs with expert annotation would be a prerequisite.

8.5 Contribution to the Sustainable Development Goals

The component contributes most directly to **SDG 3 (Good Health and Well-being)**, in particular target 3.1 on reducing maternal mortality and target 3.2 on ending preventable newborn deaths [35], by improving access to timely CTG interpretation.

It supports **SDG 10 (Reduced Inequalities)** by extending specialist-quality interpretation to mothers in remote areas, and **SDG 9 (Industry, Innovation and Infrastructure)** by demonstrating a low-cost, software-only alternative to additional monitoring hardware.

9 CONCLUSION AND FUTURE DIRECTIONS

9.1 Conclusion

This thesis described the AI-assisted CTG Interpretation Module of the Pregnancy Support System. It takes a photo of a CTG printout and produces reconstructed FHR and UC signals, CTG features with a Normal, Suspect or Pathological classification (or refuses to classify when reconstruction is inadequate). Boost resulted in a cross-validated macro-F1 of 0.917 on the known set of tabular CTG features. In terms of image reconstruction, it was shown that connected-component trace segmentation could find clinically meaningful features more accurately than a purpose-trained U-Net (mean feature error 6.56 versus 9.18) indicating thataccuracy on signals alone is insufficient to guide method selection. The paper found a failure case which existing quality metrics overlooked, and was clear that end-to-end image classification accuracy is yet to be quantified with relevant labelled data.

9.2 Achievement of objectives

Table 9.1: Achievement of objectives

Objective	Status	Evidence
O1 Pre-process and localise graph	Achieved	OpenCV detector with YOLOv8 option; IoU 0.996 in synthetic test (Sections 4.4.1, 6.2)
O2 Reconstruct FHR and UC; compare methods	Achieved	Four FHR and three UC extractors including U-Net (Sections 4.4.2–4.4.3, 6.1.2)
O3 Derive CTG features	Achieved	Eight features following FIGO thresholds; verified by unit tests (Sections 4.4.4, 5.4)
O4 Train and compare classifiers	Achieved	Four models compared; XGBoost macro-F1 0.917 (Section 6.1.1)
O5 Multi-level evaluation	Partially achieved	Signal and feature levels evaluated on images; classification evaluated on tabular data only, as image labels are unavailable
O6 Service with quality gating	Achieved	FastAPI service with three endpoints and quality states (Sections 5.2, 5.5)

9.3 Individual research and product contributions

The author's research contributions are the feature-bridge evaluation of CTG reconstruction methods, the three-level evaluation framework and the identification

of a quality-gate blind spot. The product contributions are the modular CTG analysis service, the trained classifier and U-Net, the evaluation and audit scripts and the test suite, all of which are integrated into the Pregnancy Support System through a documented API.

9.4 Future directions

- **Clinical labels for images** – collect real CTG photographs with their digital signals and expert NSP annotations, or establish a validated mapping from CTU-UHB outcomes, so that end-to-end accuracy can be measured.
- **Robust trace selection** – reject near-straight, full-width components, combine trace segmentation with the U-Net probability map, and add a plausibility check (for example, the proportion of samples at the axis limits) to the quality gate.
- **Real-photograph robustness** – add perspective correction, page-curl correction and automatic reading of the time scale and axis labels, and train detection and segmentation on real photographs.
- **Feature-schema alignment** – compute ASTV, ALTV, MSTV and MLTV in their UCI definitions and add deceleration typing relative to contractions, or retrain the classifier directly on image-derived features.
- **Explainability** – present per-prediction explanations, for example SHAP values [36], in the doctor interface.
- **Prospective evaluation** – conduct usability testing with mothers and a reader study in which obstetricians compare system output with their own interpretations.

REFERENCES

[1] World Health Organization, *WHO Recommendations on Antenatal Care for a Positive Pregnancy Experience*. Geneva, Switzerland: WHO, 2016.

[2] D. Ayres-de-Campos, C. Y. Spong, and E. Chandraharan, "FIGO consensus guidelines on intrapartum fetal monitoring: Cardiotocography," *Int. J. Gynecol. Obstet.*, vol. 131, no. 1, pp. 13–24, 2015.

[3] D. Ayres-de-Campos, J. Bernardes, A. Garrido, J. Marques-de-Sá, and L. Pereira-Leite, "SisPorto 2.0: A program for automated analysis of cardiotocograms," *J. Matern.-Fetal Med.*, vol. 9, no. 5, pp. 311–318, 2000.

[4] National Institute for Health and Care Excellence, "Fetal monitoring in labour," NICE Guideline NG229, London, U.K., Dec. 2022.

[5] J. P. Marques de Sá, J. Bernardes, and D. Ayres de Campos, "Cardiotocography," UCI Machine Learning Repository, 2000. doi: 10.24432/C51S4N.

[6] V. Chudáček, J. Spilka, M. Burša, P. Janků, L. Hruban, M. Huptych, and L. Lhotská, "Open access intrapartum CTG database," *BMC Pregnancy Childbirth*, vol. 14, Art. no. 16, 2014.

[7] A. Petrozziello, C. W. G. Redman, A. T. Papageorghiou, I. Jordanov, and A. Georgieva, "Multimodal convolutional neural networks to detect fetal compromise during labor and delivery," *IEEE Access*, vol. 7, pp. 112026–112036, 2019.

[8] Z. Zhao, Y. Zhang, Z. Comert, and Y. Deng, "Computer-aided diagnosis system of fetal hypoxia incorporating recurrence plot with convolutional neural network," *Front. Physiol.*, vol. 10, Art. no. 255, 2019.

[9] Z. Alfirevic, G. M. L. Gyte, A. Cuthbert, and D. Devane, "Continuous cardiotocography (CTG) as a form of electronic fetal monitoring (EFM) for fetal assessment during labour," *Cochrane Database Syst. Rev.*, no. 2, Art. no. CD006066, 2017.

[10] INFANT Collaborative Group, "Computerised interpretation of fetal heart rate during labour (INFANT): A randomised controlled trial," *Lancet*, vol. 389, no. 10080, pp. 1719–1729, 2017.

[11] N. Otsu, "A threshold selection method from gray-level histograms," *IEEE Trans. Syst., Man, Cybern.*, vol. 9, no. 1, pp. 62–66, 1979.

[12] K. Zuiderveld, "Contrast limited adaptive histogram equalization," in *Graphics Gems IV*, P. S. Heckbert, Ed. San Diego, CA, USA: Academic Press, 1994, pp. 474–485.

[13] A. Buades, B. Coll, and J.-M. Morel, "A non-local algorithm for image denoising," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2005, vol. 2, pp. 60–65.

[14] G. Bradski, "The OpenCV library," *Dr. Dobb's J. Softw. Tools*, 2000.

[15] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in *Med. Image Comput. Comput.-Assist. Interv. (MICCAI)*, LNCS vol. 9351, 2015, pp. 234–241.

[16] F. Milletari, N. Navab, and S.-A. Ahmadi, "V-Net: Fully convolutional neural networks for volumetric medical image segmentation," in *Proc. 4th Int. Conf. 3D Vision (3DV)*, 2016, pp. 565–571.

[17] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.

[18] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.*, 2016, pp. 785–794.

[19] Z. Cömert and A. F. Kocamaz, "Comparison of machine learning techniques for fetal heart rate classification," *Acta Phys. Pol. A*, vol. 132, no. 3, pp. 451–454, 2017.

[20] J. Spilka, V. Chudáček, M. Koucký, L. Lhotská, M. Huptych, P. Janků, G. Georgoulas, and C. Stylios, "Using nonlinear features for fetal heart rate classification," *Biomed. Signal Process. Control*, vol. 7, no. 4, pp. 350–357, 2012.

[21] A. L. Goldberger *et al.*, "PhysioBank, PhysioToolkit, and PhysioNet: Components of a new research resource for complex physiologic signals," *Circulation*, vol. 101, no. 23, pp. e215–e220, 2000.

[22] A. Georgieva *et al.*, "Computer-based intrapartum fetal monitoring and beyond: A review of the 2nd Workshop on Signal Processing and Monitoring in Labor (October 2017, Oxford, UK)," *Acta Obstet. Gynecol. Scand.*, vol. 98, no. 9, pp. 1207–1217, 2019.

[23] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 779–788.

[24] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[25] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[26] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 32, 2019, pp. 8024–8035.

[27] S. Ramírez, "FastAPI," 2018. [Online]. Available: https://fastapi.tiangolo.com

[28] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," *MIS Q.*, vol. 28, no. 1, pp. 75–105, 2004.

[29] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, "A design science research methodology for information systems research," *J. Manag. Inf. Syst.*, vol. 24, no. 3, pp. 45–77, 2007.

[30] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. 3rd Int. Conf. Learn. Represent. (ICLR)*, 2015.

[31] Parliament of the Democratic Socialist Republic of Sri Lanka, *Personal Data Protection Act, No. 9 of 2022*. Colombo, Sri Lanka, 2022.

[32] International Electrotechnical Commission, *IEC 62304:2006 Medical Device Software – Software Life Cycle Processes*. Geneva, Switzerland: IEC, 2006.

[33] International Organization for Standardization, *ISO 14971:2019 Medical Devices – Application of Risk Management to Medical Devices*. Geneva, Switzerland: ISO, 2019.

[34] International Organization for Standardization, *ISO/IEC 25010:2011 Systems and Software Engineering – Systems and Software Quality Requirements and Evaluation (SQuaRE) – System and Software Quality Models*. Geneva, Switzerland: ISO, 2011.

[35] United Nations General Assembly, "Transforming our world: The 2030 Agenda for Sustainable Development," Resolution A/RES/70/1, 2015.

[36] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 30, 2017, pp. 4765–4774.

GLOSSARY

Acceleration – A transient increase in fetal heart rate of more than 15 bpm above baseline lasting more than 15 s; generally reassuring.

Baseline FHR – The mean fetal heart rate level excluding accelerations and decelerations, normally 110–160 bpm.

Deceleration – A transient decrease in fetal heart rate of more than 15 bpm below baseline lasting more than 15 s.

Feature bridge – In this thesis, the comparison of CTG features computed from reconstructed signals with those computed from the true signals.

Macro-F1 – The unweighted mean of per-class F1-scores, giving equal weight to each class regardless of size.

Quality gate – The rule that withholds classification when the proportion of directly observed trace samples is 0.5 or lower.

Trace segmentation – Isolating the pixels belonging to a plotted curve from the background, grid and annotations of an image.

Variability – Oscillation of the fetal heart rate around its baseline; reduced variability can indicate fetal compromise.

APPENDICES

Appendix A Extended results and technical specifications

A.1 Example analysis response. The following abbreviated response illustrates the JSON returned by POST /analyze (values are illustrative):

```
{
  "success": true,
  "analysis_status": "completed",
  "graph_detected": true,
  "graph_bbox": {"x": 112, "y": 205, "width": 1690, "height": 760},
  "detection_method": "opencv",
  "detection_confidence": 0.71,
  "signal_quality": "acceptable",
  "fhr_extraction": {"method": "trace_segmentation", "valid_ratio": 0.91,
                     "quality": "acceptable"},
  "uc_extraction": {"method": "opencv_improved", "valid_ratio": 0.86,
                    "quality": "acceptable"},
  "features": {"baseline_fhr": 138.5, "short_term_variability": 1.9,
               "long_term_variability": 6.4, "accelerations": 2,
               "decelerations": 0, "mean_uc": 21.3, "max_uc": 58.0,
               "contraction_count": 3},
  "classification": {"status": "Normal", "confidence": 0.88,
                     "class_probabilities": {"Normal": 0.88,
                         "Suspect": 0.09, "Pathological": 0.03}},
  "pipeline_config": {"graph_detector": "opencv",
                      "fhr_extractor": "trace_segmentation",
                      "uc_extractor": "opencv_improved"}
}
```

A.2 Complete feature importance. Gain shares for all 21 features of the production XGBoost model, with the number of splits on each feature, are given in Table A.1.

Table A.1: Gain-based importance of all 21 input features of the production XGBoost model

Feature	Gain share (%)	Number of splits
ASTV	22.9	1382
ALTV	18.5	1009
Mean	15.7	601
AC	8.9	355
MSTV	7.5	483
LB	4.3	842
Median	3.4	440
DP	3.2	290
Mode	2.8	511
Max	2.0	467
Nmax	1.9	397
Min	1.8	605

Feature	Gain share (%)	Number of splits
Width	1.8	650
FM	1.6	445
MLTV	1.6	770
Variance	1.1	398
DL	0.5	133
Tendency	0.3	118
Nzeros	0.2	112
DS	0.0	0
DR	0.0	0

A.3 Reproducing the experiments. From the project root: *python scripts/train_classifier.py*; *python scripts/evaluate_graph_detector.py*; *python scripts/compare_fhr_extractors.py*; *python scripts/compare_uc_extractors.py*; *python scripts/train_unet_fhr.py*; *python scripts/evaluate_feature_bridge.py --compare-both*; *python scripts/evaluate_full_pipeline.py --compare-both*; *python scripts/audit_ground_truth_labels.py*; *python -m unittest tests/test_ctg_pipeline.py -v*.

Appendix B Academic integrity and AI use disclosure

Generative AI tools were used in the preparation of this thesis in accordance with the module policy.

All technical content was verified by the author against the source code, the trained model files and the evaluation reports. Every numerical result reported in this thesis was checked against the corresponding report file or test output, and every reference was checked against the original publication. The author remains responsible for the accuracy, originality and conclusions of the work.